SRI MUTHUKUMARAN INSTITUTE OF CHNOLOGY

Chikkarayapuram, Near Mangadu, Chennai-600 069.

DEPARTMENT OF INFORMATION TECHNOLOGY

FOUNDATIONS OF DATA SCIENCE USING PYTHON

LAB EXPERIMENTS :

PROGRAM USING DATA FRAMES

1. Student Marks using Data Frame

Aim
To create a **Data Frame using pandas** to store student marks and calculate **Total and Average** marks.

Software Requirements
- Python 3.x
- Pandas library

Theory
A **Data Frame** is a two-dimensional data structure in Python provided by the **pandas** library.
It stores data in **rows and columns**, similar to a table, and allows easy data manipulation and analysis.

Algorithm
1. Import the pandas library
2. Create a dictionary containing student names and marks
3. Convert the dictionary into a Data Frame
4. Display the Data Frame
5. Calculate Total marks for each student
6. Calculate Average marks
7. Display the updated Data Frame

Program
```
import pandas as pd

# Create student data
data = {
```

```
    "Name": ["Anu", "Bala", "Charan", "Divya"],
    "Maths": [85, 78, 90, 88],
    "Science": [80, 82, 95, 86],
    "English": [75, 88, 84, 90]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Student Marks DataFrame")
print(df)

# Calculate Total and Average
df["Total"] = df["Maths"] + df["Science"] + df["English"]
df["Average"] = df["Total"] / 3

print("\nUpdated DataFrame")
print(df)
```

Output (Sample):

```
Student Marks DataFrame
    Name      Maths  Science  English
0    Anu       85       80       75
1   Bala       78       82       88
2  Charan      90       95       84
3  Divya       88       86       90

Updated DataFrame
    Name     Maths      Science      English      Total      Average
0    Anu     85          80           75          240         80.00
1   Bala     78          82           88          248         82.67
2  Charan    90          95           84          269         89.67
3  Divya     88          86           90          264         88.00
```

Result

Thus, a DataFrame was successfully created to store student marks and compute Total and Average using pandas.

2.SALES AND DATA ANALYSIS USING DATAFRAME

Aim
To analyze sales data using a **pandas DataFrame** and calculate **Total Sales, Average Sales, and Highest Sales**.

Software Requirements
- Python 3.x
- Pandas library

Theory
A **DataFrame** in pandas is a two-dimensional data structure used for storing and analyzing structured data.
It is widely used in **sales analysis** to calculate totals, averages, and to identify trends.

Algorithm
1. Import pandas library
2. Create sales data using a dictionary
3. Convert data into a DataFrame
4. Display sales details
5. Calculate total sales
6. Calculate average sales
7. Find highest sales value
8. Display results

Program
```
import pandas as pd

# Create sales data
data = {
    "Product": ["Pen", "Notebook", "Pencil", "Eraser"],
    "January": [1200, 2500, 800, 400],
    "February": [1500, 2300, 900, 500],
    "March": [1700, 2600, 1000, 600]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Sales Data")
print(df)
```

```
# Calculate Total Sales
df["Total Sales"] = df["January"] + df["February"] + df["March"]

# Calculate Average Sales
df["Average Sales"] = df["Total Sales"] / 3

print("\nSales Analysis")
print(df)

# Highest sales
highest = df["Total Sales"].max()
print("\nHighest Total Sales:", highest)
```

Output (Sample)

```
Sales Data
    Product  January  February  March
0       Pen     1200      1500   1700
1  Notebook     2500      2300   2600
2    Pencil      800       900   1000
3    Eraser      400       500    600

Sales Analysis
    Product  January  February  March  Total Sales  Average Sales
0       Pen     1200      1500   1700         4400        1466.67
1  Notebook     2500      2300   2600         7400        2466.67
2    Pencil      800       900   1000         2700         900.00
3    Eraser      400       500    600         1500         500.00

Highest Total Sales: 7400
```

Result

Thus, sales data was successfully analyzed using a **pandas DataFrame** to compute **Total Sales, Average Sales, and Highest Sales**.

3.MERGING AND JOINING DATAFRAMES

Aim

To perform **merging and joining operations** on DataFrames using the **pandas** library in Python.

Software Requirements

- Python 3.x
- Pandas library

Theory

In pandas, **merging and joining** are used to combine two or more DataFrames based on a **common column or index**.

- **Merge**: Combines DataFrames using one or more common columns
- **Join**: Combines DataFrames based on index or key

Types of joins:

- Inner Join
- Left Join
- Right Join
- Outer Join

Algorithm

1. Import pandas library
2. Create two DataFrames with a common column
3. Perform inner join using merge()
4. Perform left join
5. Perform right join
6. Perform outer join
7. Display results

Program

```
import pandas as pd

# Create first DataFrame
df1 = pd.DataFrame({
    "ID": [1, 2, 3, 4],
    "Name": ["Anu", "Bala", "Charan", "Divya"]
})

# Create second DataFrame
df2 = pd.DataFrame({
    "ID": [3, 4, 5, 6],
    "Marks": [85, 90, 78, 88]
})
```

```
print("DataFrame 1")
print(df1)

print("\nDataFrame 2")
print(df2)

# Inner Join
inner_join = pd.merge(df1, df2, on="ID", how="inner")
print("\nInner Join")
print(inner_join)

# Left Join
left_join = pd.merge(df1, df2, on="ID", how="left")
print("\nLeft Join")
print(left_join)

# Right Join
right_join = pd.merge(df1, df2, on="ID", how="right")
print("\nRight Join")
print(right_join)

# Outer Join
outer_join = pd.merge(df1, df2, on="ID", how="outer")
print("\nOuter Join")
print(outer_join)
```

Output (Sample)

```
Inner Join
   ID    Name   Marks
0   3   Charan     85
1   4   Divya      90

Left Join
   ID    Name   Marks
0   1     Anu     NaN
1   2    Bala     NaN
2   3   Charan   85.0
3   4   Divya    90.0

Right Join
   ID    Name   Marks
0   3   Charan     85
```

```
1   4   Divya     90
2   5    NaN      78
3   6    NaN      88
```

Outer Join

```
   ID    Name  Marks
0   1     Anu    NaN
1   2    Bala    NaN
2   3  Charan   85.0
3   4   Divya   90.0
4   5    NaN   78.0
5   6    NaN   88.0
```

Result

Thus, **merging and joining of DataFrames** was successfully performed using different join types in pandas.

4.EMPLOYEE MANAGEMENT SYSTEM USING DATAFRAME

Aim

To design a simple Employee Management System using pandas DataFrame to store, display, and analyze employee details such as ID, Name, Department, Salary, and to perform basic operations.

Software Requirements

- Python 3.x
- Pandas library

Theory

An Employee Management System (EMS) is used to maintain employee records in an organized manner.

Using a DataFrame, employee data can be stored in tabular form and operations like salary calculation, filtering, and analysis can be easily performed.

Algorithm

1. Import pandas library
2. Create employee data using a dictionary
3. Convert the data into a DataFrame

4. Display employee details
5. Calculate total salary
6. Find average salary
7. Display employees with salary greater than a given value

Program

```
import pandas as pd

# Create employee data
data = {
    "Emp_ID": [101, 102, 103, 104],
    "Name": ["Anu", "Bala", "Charan", "Divya"],
    "Department": ["HR", "IT", "Finance", "IT"],
    "Salary": [25000, 40000, 35000, 45000]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Employee Details")
print(df)

# Total salary
total_salary = df["Salary"].sum()
print("\nTotal Salary:", total_salary)

# Average salary
avg_salary = df["Salary"].mean()
print("Average Salary:", avg_salary)

# Employees with salary > 35000
high_salary = df[df["Salary"] > 35000]
print("\nEmployees with Salary > 35000")
print(high_salary)
```

Output (Sample)

Employee Details

	Emp_ID	Name	Department	Salary
0	101	Anu	HR	25000
1	102	Bala	IT	40000
2	103	Charan	Finance	35000
3	104	Divya	IT	45000

Total Salary: 145000
Average Salary: 36250.0

Employees with Salary > 35000

	Emp_ID	Name	Department	Salary
1	102	Bala	IT	40000
3	104	Divya	IT	45000

Result

Thus, an Employee Management System was successfully implemented using a pandas DataFrame to manage and analyze employee records.

5.DATA CLEANING AND MANIPULATION USING DATAFRAME

Aim

To perform data cleaning and data manipulation operations on a dataset using pandas DataFrame.

Software Requirements

- Python 3.x
- Pandas library

Theory

Data cleaning is the process of detecting and correcting missing, duplicate, or incorrect data.

Data manipulation involves modifying data such as filtering, sorting, renaming columns, and updating values.

Pandas DataFrame provides built-in functions to perform these operations efficiently.

Algorithm

1. Import pandas library
2. Create a DataFrame with missing and duplicate values
3. Identify missing values
4. Fill or remove missing values
5. Remove duplicate records
6. Rename columns
7. Sort the DataFrame
8. Display cleaned data

Program

```
import pandas as pd

# Create raw data with missing and duplicate values
data = {
    "Emp_ID": [101, 102, 103, 103, 104],
    "Name": ["Anu", "Bala", "Charan", "Charan", None],
    "Department": ["HR", "IT", "Finance", "Finance", "IT"],
    "Salary": [25000, 40000, None, None, 45000]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Original Data")
print(df)

# Check missing values
print("\nMissing Values")
print(df.isnull())

# Fill missing salary with average salary
df["Salary"].fillna(df["Salary"].mean(), inplace=True)

# Fill missing name
df["Name"].fillna("Unknown", inplace=True)

# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Rename column
df.rename(columns={"Emp_ID": "Employee_ID"}, inplace=True)

# Sort by Salary
df.sort_values(by="Salary", inplace=True)

print("\nCleaned and Manipulated Data")
print(df)
```

Output (Sample)

Original Data

```
   Emp_ID        Name        Department                Salary
```

0	101	Anu	HR	25000.0
1	102	Bala	IT	40000.0
2	103	Charan	Finance	NaN
3	103	Charan	Finance	NaN
4	104	None	IT	45000.0

Cleaned and Manipulated Data

	Employee_ID	Name	Department	Salary
0	101	Anu	HR	25000.0
2	103	Charan	Finance	36666.7
1	102	Bala	IT	40000.0
4	104	Unknown	IT	45000.0

Result

Thus, data cleaning and manipulation was successfully performed using pandas DataFrame by handling missing values, duplicates, renaming, and sorting.